UNIPAZ
Instituto Universitario de la Paz

CLASE 2 · CORTE 1

SELECT

Base de Datos 2 · 910404 · D1

Miércoles 5 de agosto de 2026

Hermes Acevedo Castellanos

hermes.acevedo@unipaz.edu.co

ANTES DE EMPEZAR · EL CASO DEL SEMESTRE

Un libro es el título.
Un ejemplar es la copia.

La biblioteca puede tener seis copias del mismo libro, y lo que presta son **ejemplares**, no títulos. Esa distinción es la que sostiene todo el esquema con el que vamos a trabajar de aquí a noviembre.

> **Hoy le hacemos preguntas a esa base.** Y en el camino nos vamos a topar con `NULL`, que es el punto donde SQL responde algo distinto a lo que uno esperaría. Ahí nos vamos a demorar.

HOY

Tres bloques

1

La anatomía del SELECT

Qué partes tiene, y en qué orden las ejecuta **de verdad** el motor — que no es el orden en que se escriben.

2

Filtrar, ordenar, calcular

`WHERE`, operadores, expresiones, alias, `ORDER BY`, `LIMIT`, `DISTINCT`.

3

NULL

El tema real de hoy. Donde se rompen todos, incluidos los que ya trabajan. Aquí nos vamos a demorar.

Después del descanso: taller sobre la Biblioteca UNIPAZ ya cargada.

01

BLOQUE 1

La anatomía del SELECT

Seis cláusulas. El orden en que se escriben y el orden en que se ejecutan son **distintos**.

LAS SEIS CLÁUSULAS

Así se escribe. Siempre en este orden.

```
SELECT   titulo, anio_publicacion       -- qué columnas quiero
FROM     libro                          -- de dónde salen
WHERE    anio_publicacion > 2015        -- qué filas me sirven
GROUP BY …                              -- (clase 4)
HAVING   …                              -- (clase 4)
ORDER BY titulo ASC                     -- cómo las quiero ordenadas
LIMIT    10;                            -- cuántas quiero ver
```

Si cambian el orden de las cláusulas, el motor no lo perdona: es error de sintaxis. Lo único opcional es **cuáles** usan, no **en qué orden**.

LO QUE CASI NADIE LES EXPLICÓ

El motor no lo lee en ese orden.

1º `FROM` trae las filas → 2º `WHERE` descarta filas → 3º `GROUP BY` agrupa → 4º `HAVING` descarta grupos → 5º `SELECT` calcula columnas → 6º `ORDER BY` ordena → 7º `LIMIT` recorta

Esto explica dos cosas que les van a pasar hoy: por qué un alias que definen en el `SELECT` no se puede usar en el `WHERE` (todavía no existe), y por qué **sí** se puede usar en el `ORDER BY` (ya existe).

LA CONSECUENCIA, EN VIVO

El alias no existe cuando corre el `WHERE`

NO FUNCIONA

```
SELECT titulo,
       anio_publicacion AS anio
FROM   libro
WHERE  anio > 2015;
```

ERROR: column "anio" does not exist. El `WHERE` corrió **antes** del `SELECT`.

No es un capricho del lenguaje. Es el orden de ejecución hecho visible.

SÍ FUNCIONA

```
SELECT titulo,
       anio_publicacion AS anio
FROM   libro
WHERE  anio_publicacion > 2015
ORDER BY anio DESC;
```

El `ORDER BY` corre **después** del `SELECT`: ahí el alias ya nació.

02

BLOQUE 2

Filtrar, ordenar, calcular

Todo sobre la Biblioteca UNIPAZ. Los libros son los que están de verdad en el estante.

✋ PREGUNTA AL GRUPO

WHERE · EL FILTRO

Libros publicados después de 2015, ordenados por título

```
SELECT titulo,
       anio_publicacion
FROM   libro
WHERE  anio_publicacion > 2015
ORDER BY titulo;
```

RESULTADO

titulo	anio
Bases de datos: teoría y práctica…	2025
Modelado de datos con UML	2018
SQL práctico para desarrolladores	2019

3 filas · de 9 libros

Pregunta al aire: hay un libro en la tabla cuyo año es `NULL`. ¿Salió? ¿Debería haber salido? Guarden la respuesta — es el bloque 3.

WHERE · OPERADORES

Los que van a usar todo el semestre

OPERADOR	EJEMPLO SOBRE LA BIBLIOTECA	QUÉ HACE
`= <> < > <= >=`	`anio_publicacion >= 2013`	Comparación directa. `<>` es «distinto de» (también vale `!=`).
`BETWEEN`	`anio_publicacion BETWEEN 2012 AND 2015`	Rango **inclusivo** en los dos extremos. Ojo con eso.
`IN`	`estado IN ('prestado','deteriorado')`	Pertenece a una lista. Más limpio que tres `OR`.
`LIKE`	`titulo LIKE '%datos%'`	`%` = cualquier cantidad de caracteres · `_` = exactamente uno.
`ILIKE`	`titulo ILIKE '%SQL%'`	Igual que `LIKE` pero **ignora mayúsculas**. Es de PostgreSQL, no del estándar.
`AND OR NOT`	`tipo='docente' AND programa IS NOT NULL`	Combinan condiciones. `AND` se evalúa antes que `OR`: usen paréntesis.

LIKE · BUSCAR POR PATRÓN

Dos comodines. Eso es todo lo que hay que aprender.

`%` = cualquier cantidad de caracteres (incluso ninguno) · `_` = exactamente un carácter.
Buscando sobre los títulos de nuestros 9 libros:

`'%datos%'`	**7 de los 9 libros.** Diseño de bases de **datos** · Bases de **datos** · Gestión de bases de **datos** · Modelado de **datos** con UML · y 3 más. Se salvan solo «...sistemas NoSQL» y «SQL práctico...».
`'Bases%'`	**3 libros.** Solo los que **empiezan** por ahí: **Bases** de datos · **Bases** de datos: desarrollo... · **Bases** de datos: teoría...
`'%web'`	**1 libro.** El que **termina** exactamente en eso: «...multiplataforma y **web**»
`'_ases%'`	**3 libros.** Un carácter cualquiera + «ases»: **Bases**... — sirve cuando no saben si va con mayúscula o hay un error de digitación
`'%DATOS%'`	**0 libros.** `LIKE` **distingue mayúsculas**. Para ignorarlas: `ILIKE '%DATOS%'` → vuelven los 7.

EXPRESIONES Y ALIAS

El SELECT no solo trae columnas: también calcula

```
SELECT id_prestamo,
       fecha_devolucion_esperada
       - fecha_prestamo AS dias_plazo,
       fecha_prestamo + 30 AS tope_maximo
FROM   prestamo
ORDER BY dias_plazo DESC
LIMIT  4;
```

RESULTADO

id	dias_plazo	tope_maximo
1	14	2026-07-02
2	14	2026-07-03
3	14	2026-07-10
4	14	2026-07-15

el plazo es de 14 días para todos

Restar dos DATE en PostgreSQL da un **número de días**. El AS le pone nombre a la columna calculada — sin alias, PostgreSQL la llamaría algo ilegible.

ORDER BY

Ordenar por varias columnas: la segunda desempata a la primera

```
SELECT nombre, tipo, programa
FROM   usuario
ORDER BY tipo ASC,
         nombre ASC;
```

RESULTADO (PRIMERAS FILAS)

nombre	tipo
Gloria Patricia Méndez	docente
Hermes Acevedo	docente
Wilson Rodríguez	docente
Andrés Felipe Guzmán	estudiante
Carlos Eduardo Vega	estudiante
Sandra Milena Ávila	NULL

ASC es el valor por omisión: si no escriben nada, ordena ascendente.
Fíjense en la última fila — hay un usuario sin tipo, y quedó al final.
Volvemos a eso.

DISTINCT

Quitar repetidos

```
SELECT estado
FROM   ejemplar;
```

20 filas, con los valores repetidos una y otra vez.

```
SELECT DISTINCT estado
FROM   ejemplar;
```

estado
disponible
prestado
deteriorado
NULL
4 filas

> **Ahí está otra vez.** Aparece un `NULL` entre los estados. Y hay **dos** ejemplares sin catalogar, pero `DISTINCT` los mostró como **uno**. Retengan esa rareza.

03

BLOQUE 3 · EL TEMA DE HOY

NULL

Aquí se rompen todos. Estudiantes, egresados y gente que ya lleva años trabajando.

ANTES DE NADA

`NULL` no es cero. Tampoco es vacío.

`0`	`''`	`NULL`
Un número. Vale cero. **Sé** que no debe nada.	Una cadena vacía. **Sé** que el texto está en blanco.	**No sé.** Ausencia de dato. No hay valor que comparar.

`NULL` no es un valor: es la **marca de que no hay valor**.

Todo lo raro que sigue sale de ahí.

✋ PARE AQUÍ · QUE PREDIGAN

LA PREGUNTA QUE PARECE FÁCIL

¿Cuántos ejemplares no están prestados?

```
SELECT COUNT(*) FROM ejemplar;                          -- ¿cuántos hay en total?
SELECT COUNT(*) FROM ejemplar WHERE estado =  'prestado';  -- ¿cuántos prestados?
SELECT COUNT(*) FROM ejemplar WHERE estado <> 'prestado';  -- ¿cuántos NO prestados?
```

Escríbanlo en el cuaderno antes de que lo corra: **¿cuánto tiene que dar la suma de los dos últimos?**

EL RESULTADO REAL

La cuenta no cuadra

7
estado = 'prestado'

+

11
estado <> 'prestado'

=

18
y la tabla tiene **20 ejemplares**

→

2
ejemplares desaparecidos

> **Dos ejemplares se esfumaron** y la base no dio ni un error ni una advertencia. Son los dos que llegaron por donación y nadie catalogó: tienen `estado` en `NULL`.

Si esto fuera un inventario real, acabamos de perder dos libros del informe. Sin enterarnos.

FILA POR FILA · LO QUE HIZO EL MOTOR

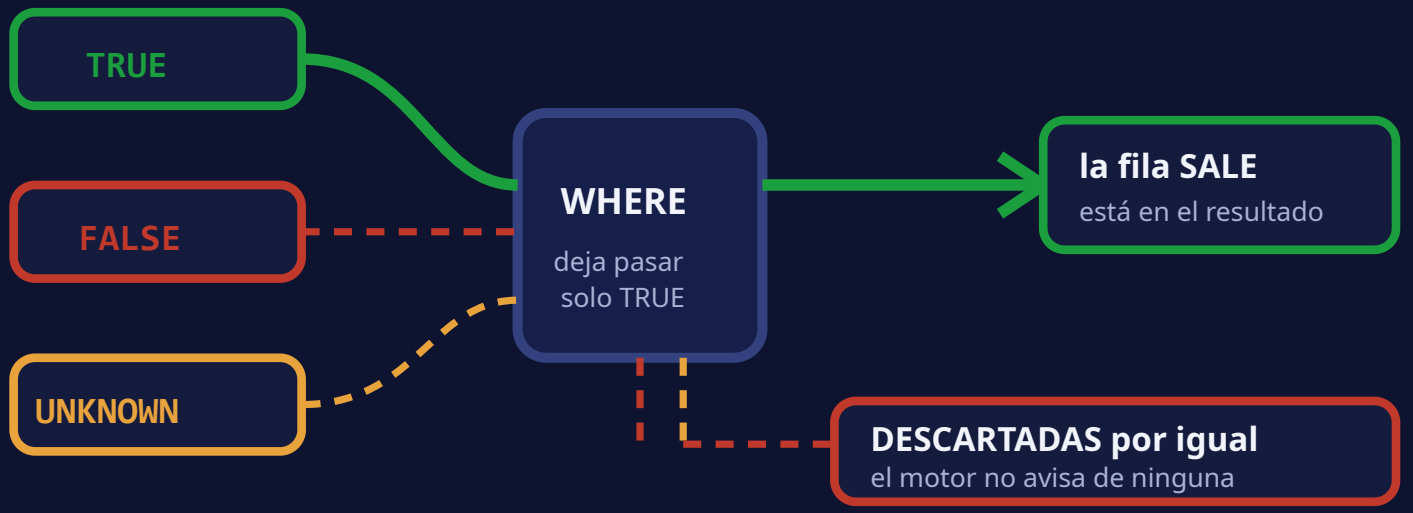
Ninguna consulta se equivocó. Miren las dos últimas filas.

código	estado	estado = 'prestado'	estado <> 'prestado'	¿en qué consulta sale?
BU-0002	prestado	TRUE	FALSE	en la 1ª
BU-0003	prestado	TRUE	FALSE	en la 1ª
BU-0001	disponible	FALSE	TRUE	en la 2ª
BU-0004	deteriorado	FALSE	TRUE	en la 2ª
BU-0019	NULL	UNKNOWN	UNKNOWN	en NINGUNA ❌
BU-0020	NULL	UNKNOWN	UNKNOWN	en NINGUNA ❌

Para las filas con `estado` en `NULL`, **las dos condiciones dieron lo mismo**: `UNKNOWN`. Y como ninguna dio `TRUE`, esas dos filas **no entraron en ningún resultado**. No hay error: hay una tercera respuesta posible que nadie les había contado.

EL WHERE ES UN COLADOR

Y solo tiene un agujero: el de TRUE.



Aquí está toda la clase en una imagen. La intuición dice que el colador tiene dos salidas —«cumple» y «no cumple»— y que toda fila cae en una de las dos. En realidad tiene **tres entradas y una sola salida**: lo que no es `TRUE` se cae, y `UNKNOWN` se cae por el mismo hueco que `FALSE`.

POR QUÉ PASÓ

Comparar con lo desconocido no da verdadero. Da desconocido.

```
estado = 'prestado'   → UNKNOWN
estado <> 'prestado'  → UNKNOWN
estado = estado       → UNKNOWN
NULL   = NULL         → UNKNOWN
```

Cuando uno de los lados es `NULL`, el resultado **nunca** es verdadero ni falso.

Y la regla que lo explica todo:
el `WHERE` solo deja pasar las filas donde la condición es **TRUE**.

FALSE no pasa. **UNKNOWN** tampoco.

Por eso los dos ejemplares sin catalogar no salieron ni en la consulta de «prestados» ni en la de «no prestados». Quedaron en el limbo.

LÓGICA DE TRES VALORES

SQL no tiene dos valores de verdad. Tiene tres.

AND

	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR

	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

Las dos casillas que valen oro: `FALSE AND UNKNOWN` = **FALSE** (si ya es falso, da igual lo que no sepamos) y `TRUE OR UNKNOWN` = **TRUE**. En los demás cruces la incertidumbre **se contagia**.

LA FORMA CORRECTA

`IS NULL`, nunca `= NULL`

MIENTE EN SILENCIO

```
SELECT COUNT(*)
FROM   prestamo
WHERE  fecha_devolucion_real = NULL;
```

count
0

Cero. Y no es que no haya: es que la pregunta está mal hecha. Ni un error, ni una advertencia.

`IS NULL` e `IS NOT NULL` no son comparaciones: son **pruebas de existencia**. Por eso sí devuelven `TRUE` o `FALSE`, nunca `UNKNOWN`.

PREGUNTA DE VERDAD

```
SELECT COUNT(*)
FROM   prestamo
WHERE  fecha_devolucion_real IS NULL;
```

count
7

Siete libros están fuera de la biblioteca ahora mismo. Ese es el dato que el bibliotecario necesita.

EL CONTAGIO

Cualquier operación con NULL devuelve NULL

```
-- aritmética
5 + NULL              → NULL
100 * NULL            → NULL
dias_mora * 500       → NULL  -- la multa se evapora

-- concatenación de texto
'Hola ' || NULL       → NULL
nombre || nacionalidad→ NULL  -- se pierde el nombre ent
```

PASA DE VERDAD, CON NUESTROS DATOS

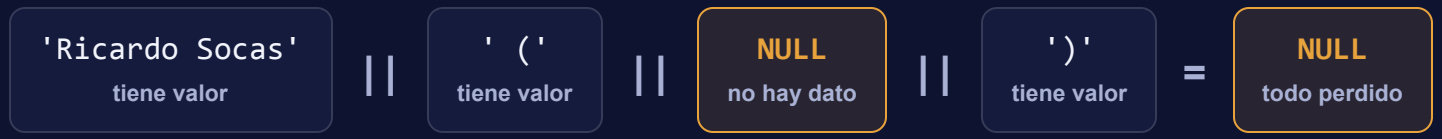
```
SELECT nombre || ' (' ||
       nacionalidad || ')' AS ficha
FROM   autor;
```

ficha
Paloma Martínez Fernández (España)
Jaime Ospino Rivas (Colombia)
NULL
Eduardo Jorge Reinosa (Argentina)

Ricardo Socas Gutiérrez desapareció. No salió «Socas ()». Salió NULL: un dato faltante se comió el nombre completo.

CÓMO SE PROPAGA, PASO A PASO

Un solo `NULL` contamina toda la expresión



> **Tres de los cuatro pedazos tenían valor** — y el resultado igual salió vacío. `NULL` no se comporta como un elemento neutro: **absorbe**. Es la misma lógica del `UNKNOWN`: si un pedazo se desconoce, el todo se desconoce.

Piénsenlo así: si no saben la nacionalidad, tampoco saben cómo se ve la ficha completa. La base es honesta — el problema es que su honestidad borra el nombre.

LA HERRAMIENTA

COALESCE

Recibe varios valores y devuelve **el primero que no sea `NULL`**.
Es la forma de decidir qué poner cuando no hay dato.

```
SELECT nombre || ' (' ||
       COALESCE(nacionalidad,
                'sin dato') || ')' AS ficha
FROM   autor;
```

ficha
Paloma Martínez Fernández (España)
Ricardo Socas Gutiérrez (sin dato)
Eduardo Jorge Reinosa (Argentina)

10 filas · ninguna perdida

Pero cuidado: `COALESCE` no «arregla» nada — **decide**. Poner `COALESCE(dias_mora, 0)` es afirmar que «sin dato» significa «no debe nada». Eso puede ser cierto o puede ser una mentira. Es una decisión del negocio, no de la sintaxis.

Y SU HERMANO

`NULLIF` hace lo contrario: crea un `NULL`

```
-- si los dos son iguales → NULL
NULLIF(0, 0)        → NULL
NULLIF(5, 0)        → 5

-- el uso clásico: evitar división por cero
SELECT total /
       NULLIF(cantidad, 0)
FROM   ...;
```

Si `cantidad` es 0, `NULLIF` lo vuelve `NULL`, y dividir por `NULL` da `NULL` — en lugar de reventar la consulta con **division by zero**.

Es el mismo razonamiento del punto 7 del diagnóstico de otra asignatura: corregir un error puede **abrir** otro. Aquí se cierra a propósito.

EL SEGUNDO GOLPE

El promedio que miente.

```
SELECT AVG(dias_mora) FROM prestamo;   -- "el promedio de mora de la biblioteca"
```

7,25
lo que devuelve la consulta

vs

2,90
el promedio real sobre los 20 préstamos

Las funciones de agregación ignoran los `NULL`. `AVG` sumó solo los **8** préstamos que tenían un número en `dias_mora` y dividió por 8 — no por 20. El resultado no es «el promedio de la biblioteca»: es **el promedio de los morosos**.

Ese número, puesto en un informe para el rector, es una mentira con firma.

CÓMO SE VE EL ENGAÑO

COUNT(*) y COUNT(columna) no son lo mismo

```
SELECT COUNT(*)          AS filas,
       COUNT(dias_mora)  AS con_dato,
       AVG(dias_mora)    AS promedio_falso,
       AVG(COALESCE(dias_mora,0))
                         AS promedio_real
FROM   prestamo;
```

filas	con_dato	falso	real
20	8	7,25	2,90

`COUNT(*)` cuenta **filas**. `COUNT(columna)` cuenta **valores no nulos** de esa columna. La diferencia entre 20 y 8 es exactamente el tamaño del engaño.

Truco de oficio: cuando un promedio les parezca raro, comparen `COUNT(*)` con `COUNT(la_columna)`. Si no coinciden, ya saben dónde está el problema.

VÉANLO CON LA COLUMNA EN LA MANO

Lo que AVG sumó, y lo que ignoró

DIAS_MORA · 20 FILAS

NULL
0
NULL
12
18
NULL
5
NULL

… y 12 más

Las celdas punteadas no valen cero. Están vacías. AVG no las trató como 0 — **las sacó del cálculo**, y también del divisor.

58 / 8
lo que hizo
= 7,25

≠

58 / 20
lo que el informe decía
= 2,90

Misma suma arriba. **Divisor distinto.** Ahí está el 150% de diferencia.

LA INCOHERENCIA DEL ESTÁNDAR

Para = dos `NULL` son distintos. Para `DISTINCT`, iguales.

```
NULL = NULL    → UNKNOWN
                 (no son iguales)
```

Como comparación, dos datos desconocidos **no** se consideran el mismo.

```
SELECT DISTINCT estado
FROM   ejemplar;
→ 4 filas, con UN solo NULL
  (aunque hay DOS sin catalogar)
```

Como agrupación, `DISTINCT` y `GROUP BY` sí los juntan en uno.

> **No hay que justificarla, hay que conocerla.** Es una inconsistencia real del estándar SQL, y está en todos los motores. Si la saben, no los sorprende en un informe.

ORDER BY Y LOS NULL

En PostgreSQL, NULL se ordena como si fuera el valor más grande

```
ORDER BY anio_publicacion ASC
   → los NULL quedan de ÚLTIMOS

ORDER BY anio_publicacion DESC
   → los NULL quedan de PRIMEROS

-- y se controla a mano:
ORDER BY anio_publicacion DESC
         NULLS LAST;
```

DESC SIN CONTROLAR

titulo	anio
Introducción a los sistemas NoSQL	NULL
Bases de datos: teoría y práctica…	2025
SQL práctico para desarrolladores	2019

El libro «más nuevo» del listado resultó ser el que **no tiene año**. Un top 10 armado así arranca con basura.

EL ATAJO ELEGANTE

`IS DISTINCT FROM`: comparar tratando `NULL` como un valor más

LARGO PERO CORRECTO

```
WHERE estado <> 'prestado'
   OR estado IS NULL
```

Funciona. Es lo que van a escribir el 90% de las veces, y está bien.

CORTO Y CORRECTO

```
WHERE estado
      IS DISTINCT FROM 'prestado'
```

Idéntico resultado: **13 filas**. Aquí `NULL` sí cuenta como «distinto de prestado».

Las dos devuelven los 13 ejemplares que no están prestados — incluidos los 2 sin catalogar. La cuenta por fin cuadra: **7 + 13 = 20**.

PARA EL CUADERNO

Las seis reglas de `NULL`

1. `NULL` no es 0 ni `''`. Es **ausencia de dato**.
2. Se pregunta con `IS NULL`. **Nunca** con `= NULL`.
3. El `WHERE` solo deja pasar `TRUE`. `UNKNOWN` **se descarta.**
4. Cualquier cálculo o concatenación con `NULL` da `NULL`.
5. Los agregados **ignoran** los `NULL`. `COUNT(*)` ≠ `COUNT(col)`.
6. `COALESCE` no arregla: **decide**. Y decidir es del negocio.

> **La pregunta que hay que hacerse siempre, antes de escribir el `WHERE`:** ¿esta columna puede venir en `NULL`? Si la respuesta es sí, la consulta necesita decidir qué hacer con esas filas. **No decidir también es una decisión** — y es la que pierde los dos ejemplares.

04

3:00 – 4:30 P.M.

Taller

Ocho puntos sobre la Biblioteca UNIPAZ.
Seis de piso, dos de reto.

CÓMO SE TRABAJA

En parejas.
Y se entrega hoy.

Piso · puntos 1 a 6

Obligatorios. Si los seis quedan bien, el taller está aprobado. Nada de esto se resuelve copiando: hay que decidir qué hacer con los `NULL`.

Techo · puntos 7 y 8

Opcionales, suben nota. Son de **criterio**: no piden escribir una consulta, piden explicar por qué una consulta está mintiendo.

El enunciado está en el repositorio de la asignatura — no hay hoja que se pierda. Se entrega el archivo `.sql` con las consultas **y los comentarios**: una consulta sin explicación no puntúa completo, porque aquí importa por qué la escribieron así.

La próxima clase arranca con **dos defensas orales al azar** de este taller. Dos minutos cada una.

UNIPAZ
Instituto Universitario de la Paz

La próxima semana: `JOIN`.

Cómo unir tablas — y por qué un `LEFT JOIN` con una condición mal puesta deja de ser un `LEFT JOIN`.

Hermes Acevedo

Base de Datos 2 · 910404 · D1 · 2026B